Normalización más allá de la 1NF

# Introducción

Una vez entendidas las formas normales y con el punto inicial cumplido, es decir, una vez que hemos alcanzado con una tabla la primera forma normal, estamos accediendo a un diseño más estable y flexible, lo que nos queda por ver es la aplicación de las restantes formas normales, y lo haremos con ejemplos sencillos que nos permitirán entender el proceso completo y aplicarlo en nuestros proyectos.

## 1. Normalización más allá de la 1NF

Cualquier tabla que esté en la segunda forma normal (2NF) o posterior, también está, por definición, en 1NF (cada forma normal tiene criterios más rigurosos que su precursora). Por una parte, una tabla que está en 1NF puede o no puede estar en 2NF; si está en 2NF, puede o no puede estar en 3NF, y así sucesivamente.

Las formas normales posteriores a la 1NF son pensadas para ocuparse de las situaciones en las que una tabla sufre de problemas de diseño que pueden comprometer la integridad de los datos dentro de ella. Por ejemplo, la tabla siguiente está en 1NF, pero no está en 2NF y, por lo tanto, es vulnerable a inconsistencias lógicas.

**Tabla 1: Tabla *MailSubscriptores*, con columna teléfono ampliada**

| Id subscriptor | DirecciónMail | Apellido | nombre |
|---|---|---|---|
| 108 | juan@mail.com | Juan | Rodríguez |
| 252 | franco@direccion.com | Franco | Hernández |
| 252 | josefina@hotmail.com | Josefina | López |
| 360 | marina@mailing.com | María | Zapata |

Fuente: elaboración propia.

La clave primaria de la tabla es {ID del subscriptor, Dirección de correo}.

Si Carol Robertson cambia su apellido (Robertson) por el de matrimonio (Hall), el cambio debe ser aplicado a dos filas. Si el cambio es aplicado solamente a una

fila, resulta en una contradicción: la pregunta, ¿cuál es nombre del cliente 252? tiene dos respuestas que están en conflicto. La 2NF aborda este problema.

**Ejemplo 1 (2FN): considera una tabla que describe las habilidades de los empleados.**

**Tabla 2: Tabla *HabilidadesXempleados***

| Empleado | Habilidad | Lugar actual de trabajo |
|---|---|---|
| González | Mecánico | Calle 3 |
| González | Carpintero | Calle 3 |
| González | Pintor | Calle 3 |
| Pérez | Orfebre | Calle 59 |
| Juárez | Zapatero | Calle 59 |
| Juárez | Constructor | Calle 59 |

Fuente: elaboración propia.

La clave primaria de la tabla es {Empleado, Habilidad}. El atributo *Lugar actual de trabajo* depende de la clave candidata *Empleado*. Esto nos indicará que la tabla no está en 2NF, cuestión que podemos ver, por ejemplo, cuando se repiten las direcciones de las personas. Los datos finales resultarán contradictorios.

Asimismo, podemos afirmar que no todas las tablas 2NF estarán libres de anomalías de actualización.

Un ejemplo de una tabla 2NF que sufre de anomalías de actualización es la tabla 3.

**Tabla 3: Tabla *GanadoresdeTorneos***

| Torneo | Año | Ganador | Fecha de nac. del ganador |
|---|---|---|---|
| Tenis open | 1990 | Federico G. | 12 de julio de 1975 |
| Futbol open | 1992 | Martín A | 15 de agosto de 1989 |
| Vóley open | 1985 | José S. | 16 de septiembre de 1993 |
| Running open | 2005 | Sergio L. | 21 de diciembre de 1990 |
| Swimming open | 1996 | Agustín H. | 14 de marzo de 1977 |

Fuente: elaboración propia.

Las categorías *ganador* y *fecha de nacimiento* están determinadas por una clave completa pero no son parte de ella. Ambas resultan redundantes en múltiples registros.

Una tabla o relación que tiene una clave primaria simple está en 2FN por no poder existir una dependencia parcial con parte de la clave, porque esta tiene un solo atributo o columna.

Por lo que, en tablas donde la clave primaria es compuesta, es necesario realizar el análisis de que los atributos no clave dependan de toda la clave primaria compuesta.

La tercera forma normal (3NF) se utiliza para la normalización de las bases de datos. Solo estará en 3NF sí: a) la tabla está en la segunda forma normal (2NF), b) ningún atributo no primario de la tabla es dependiente transitivamente de una clave candidata.

**Ejemplo**

**Tabla 4: Una tabla en 2FN y no en la 3FN GanadoresdelTorneo**

| Torneo | Año | Ganador | Fecha de nac. del ganador |
|---|---|---|---|
| Tenis open | 1990 | Federico G. | 12 de julio de 1975 |
| Futbol open | 1992 | Martín A | 15 de agosto de 1989 |
| Vóley open | 1985 | José S. | 16 de septiembre de 1993 |
| Running open | 2005 | Sergio L. | 21 de diciembre de 1990 |

Fuente: elaboración propia.

La clave primaria es {Torneo, Año}.

La violación de la 3NF ocurre porque el atributo no primario Fecha de *nacimiento* del ganador es dependiente transitivamente de {Torneo, Año}, vía el atributo no primario *Ganador*. El hecho de que la *Fecha de nacimiento* del ganador es funcionalmente dependiente en el *Ganador* hace la tabla vulnerable a inconsistencias lógicas, pues no hay nada que impida a la misma persona ser mostrada con diferentes fechas de nacimiento en diversos registros.

Para expresar los mismos hechos sin violar la 3NF, es necesario dividir la tabla en dos (tabla 5).

**Tabla 5: La solución en la 3FN requiere dos tablas GanadoresdelTorneo**

| Torneo | Año | Ganador |
|---|---|---|
| Tenis open | 1990 | Federico G. |
| Fútbol open | 1992 | Martín A |
| Vóley open | 1985 | José S. |
| Running open | 2005 | Sergio L. |

Fuente: elaboración propia

**Tabla 6. Información del jugador**

| Jugador | Fecha de nacimiento |
|---|---|
| Federico G. | 12 de julio de 1975 |
| Martín A | 15 de agosto de 1989 |
| José S. | 16 de septiembre de 1993 |

Fuente: elaboración propia.

Las anomalías de actualización no pueden ocurrir en estas tablas, las cuales están en 3NF.

**Ejemplo paso a paso**

> Digamos que queremos crear una tabla con la información de usuarios, y los datos que se deben guardar son el nombre, la empresa, la dirección de la empresa y algún *e-mail* o bien URL, si las tienen. En principio, se encuentra en una situación como la siguiente y define la estructura de una tabla como esta. (Wise, s. f., https://www.lawebdelprogramador.com/temas/tecdiseno.php).

**Tabla 7: Normalización cero en la tabla *Usuarios***

| Nombre | Empresa | Dirección empresa | url1 | url2 |
|---|---|---|---|---|
| Joe | ABC | 1 Work Lane | abc.com | xyz.com |
| Jill | XYZ | 1 Job Street | abc.com | xyz.com |

Fuente: Wise, s. f., https://www.lawebdelprogramador.com/temas/tecdiseno.php

> Diríamos que la anterior tabla está en nivel de normalización cero porque ninguna de nuestras reglas de normalización ha sido aplicada. Observa los campos url1 y url2. ¿Qué haremos cuando en nuestra aplicación necesitemos una tercera URL? ¿Quieres tener que añadir otro campo/columna a tu tabla y tener que reprogramar toda la entrada de datos de tu código PHP? Se quiere crear un sistema funcional que pueda crecer y adaptarse fácilmente a los nuevos requisitos. Aplicaremos las reglas del primer nivel de formalización-normalización. (Wise, s. f., https://www.lawebdelprogramador.com/temas/tecdiseno.php).

## 2. Primera forma normal (1FN)

1. “Eliminar los grupos repetitivos de las tablas.
2. Crear una tabla separada por cada grupo de datos relacionados.
3. Identificar cada grupo de datos relacionados con una clave primaria” (Wise, s. f., https://www.lawebdelprogramador.com/temas/tecdiseno.php).

Es notable que la tabla está rompiendo la primera regla cuando repetimos los campos url1 y url2. ¿Y qué pasa con la tercera regla, la clave primaria? Las columnas definidas no permiten identificar cada fila, porque ¿qué pasaría si tuviéramos dos usuarios llamados Joe y queremos diferenciarlos? Para este problema, se puede definir una clave artificial que se puede implementar en los motores de base de datos con un campo autoincrementable o con el uso de un objeto secuencia (Wise, s. f.). Veamos cómo quedó la tabla.

**Tabla 8: Primera forma normal (1FN) en la tabla *Usuarios***

| **userId** | Nombre | empresa | dirección _ empresa | url |
|---|---|---|---|---|
| 1 | Joe | ABC | 1 Work Lane | abc.com |
| 1 | Joe | ABC | 1 Work Lane | xyz.com |
| 2 | Jill | XYZ | 1 Job Street | abc.com |
| 2 | Jill | XYZ | 1 Job Street | xyz.com |

Fuente: Wise, s. f., https://www.lawebdelprogramador.com/temas/tecdiseno.php

> Ahora diremos que nuestra tabla está en el primer nivel de F/N. Hemos solucionado el problema de la limitación del campo *url*. Pero, sin embargo, vemos otros problemas. Cada vez que introducimos un nuevo registro en la tabla *usuarios*, tenemos que duplicar el nombre de la empresa y del usuario. No solo nuestra BD crecerá muchísimo, sino que será muy fácil que la BD se corrompa si escribimos mal alguno de los datos redundantes. (Wise, s. f., https://www.lawebdelprogramador.com/temas/tecdiseno.php).

Como la tabla tiene una clave primaria simple, podemos decir que está en 2FN, aunque con problemas. La solución para esto es la aplicación de la tercera forma normal (3FN).

Usaremos otro ejemplo para aplicar la segunda forma normal (tabla 9).

**Tabla 9: Tabla pedidosClientesProd**

| clienteId | Prod_id | cantidad | Precio | Nombre cliente | Fechapedido |
|---|---|---|---|---|---|
| 1 | P15 | 5 | 1,75 | Cliente1 | 01/11/2020 |
| 1 | P16 | 8 | 2,55 | Cliente1 | 10/11/2020 |
| 2 | P15 | 10 | 1,75 | Cliente2 | 01/09/2020 |
| 2 | P18 | 8 | 5,00 | Cliente2 | 07/08/2020 |

Fuente: Elaboración propia.

En este caso, la tabla *pedidosclientesProd* ha sido diseñada, provisoriamente, con una clave primaria compuesta por el id del cliente que pide y el id del producto pedido. Consideramos que el precio se fija por la oportunidad del pedido, tomando como base el precio del producto en su tabla, pero que puede variar por alguna promoción o negociación en el pedido, por lo que se decidió que depende efectivamente de la combinación de las dos columnas de la clave primaria. El problema es que llegamos con la tabla que no puede cumplir con la 2FN porque la columna *Nombre de cliente* que depende solamente de una parte de la clave primaria, *clienteId*. Sí que debe ser eliminada y, si no existiera, crear la tabla *clientes* con la clave *clienteId* y el atributo que corresponde sacar, *nombre cliente*, de manera que quedan las tablas como se ve en la tabla 10.

## 3. Segunda forma normal

1. Crear una tabla aparte con los datos que no dependen de la clave como URL (depende de *empresa*).
2. Relacionar estas tablas mediante una clave foránea.

**Tabla 10: Aplicación de la 2FN en la tabla pedidosClientesProd agregando otra tabla Clientes**

| clienteId | Prod_id | cantidad | Precio | Fechapedido |
|---|---|---|---|---|
| 1 | P15 | 5 | 1,75 | 01/11/2020 |
| 1 | P16 | 8 | 2,55 | 10/11/2020 |
| 2 | P15 | 10 | 1,75 | 01/09/2020 |
| 2 | P18 | 8 | 5,00 | 07/08/2020 |

| clienteId | Nombre cliente |
|---|---|
| 1 | Cliente1 |
| 2 | Cliente2 |

Fuente: Elaboración propia.

Hemos creado tablas separadas y la columna de la clave primaria de *pedidosClientesProd*, *clienteId* está relacionando ahora como la clave foránea con la tabla *clientes*, que podría estar previamente creada, lo nuevo es la clave foránea y la eliminación de la columna no dependiente de toda la clave, como pide la 2FN.

## 4. Tercera forma normal (3FN)

Retomamos la tabla de *Usuarios* de la tabla 8, en esta notábamos que se repite toda la fila para cada URL nueva que tenga la empresa, es decir que tiene columnas no clave (URL) que dependen funcionalmente de otra columna no clave (*emprId*).

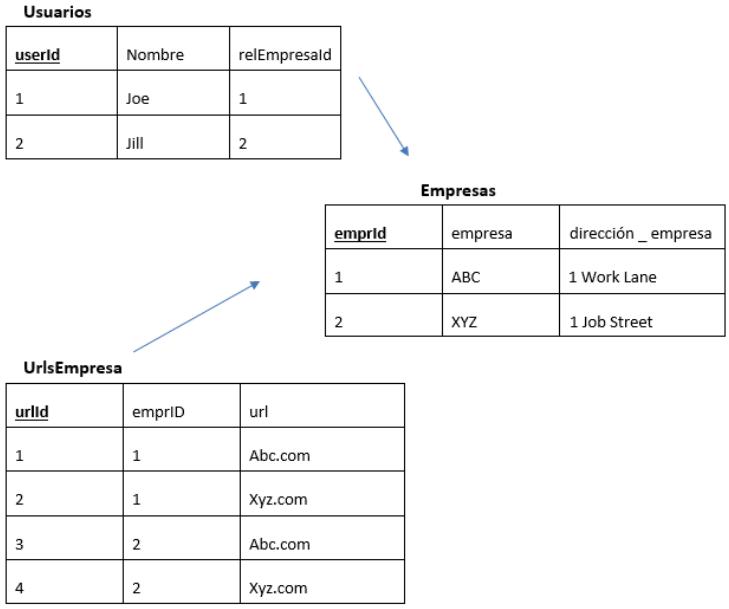
Pasos para que quede en la 3FN:

1. Crear otra tabla y llevar a aquellos campos que no dependan de la clave y definir las respectivas claves foráneas.

Nuestro nombre de empresa y su dirección no tienen nada que ver con el campo *userId*, así que tienen que tener su propia tabla y conectar a *empresaId* como

clave foránea a la tabla *empresa*, y otra tabla para las varias URL de cada empresa, y una clave foránea del lado de *UrlsEmpresa* que referencie también a Empresas.

**Tabla 11: Normalización a 3FN en la tabla *Usuarios***



**Usuarios**

| userId | Nombre | relEmpresaId |
|---|---|---|
| 1 | Joe | 1 |
| 2 | Jill | 2 |

**Empresas**

| emprId | empresa | dirección _ empresa |
|---|---|---|
| 1 | ABC | 1 Work Lane |
| 2 | XYZ | 1 Job Street |

**UrlsEmpresa**

| urlId | emprID | url |
|---|---|---|
| 1 | 1 | Abc.com |
| 2 | 1 | Xyz.com |
| 3 | 2 | Abc.com |
| 4 | 2 | Xyz.com |

Fuente: Elaboración propia.

Ahora tenemos la clave primaria *emprId*, en la tabla *empresas*, relacionada con la clave externa *recEmpresaId* en la tabla usuarios, y podemos añadir 200 usuarios mientras que solo tenemos que insertar el nombre 'ABC' una vez. Nuestras tablas de *usuarios* y *url* pueden crecer todo lo que quieran sin duplicación ni corrupción de datos. Es normalmente aceptado que el tercer nivel de forma normal es suficiente.

## ¿Cuál es la forma normal en la que está una tabla que no tiene grupos de columnas repetitivas?

En todas las formas normales se cumple

Primera forma normal exclusivamente

Segunda forma normal exclusivamente

Tercera forma normal exclusivamente

En las dos primeras exclusivamente

**La siguiente afirmación, ¿a qué forma normal se aplica?: Ningún atributo no clave depende de otro atributo no clave.**

La tercera forma normal

La segunda forma normal

La primera forma normal

La forma normal de Boyce-codd

# Referencias

**Wise, B.** (s. f.). Normalización de bases de datos y técnicas de diseño. Recuperado de https://www.lawebdelprogramador.com/temas/tecdiseno.php